SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: **Database Management Systems**

COURSE CODE: **CSA05**

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.* (BL3, BL4)

Activity Tool : Industry Problem Solving Task (Medium)

Assessment Tool Weightage: 35%

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.	BL4 – Analyze	**5**
2	A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.	BL4 – Analyze	**5**
3	A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.	BL4 – Analyze	**5**

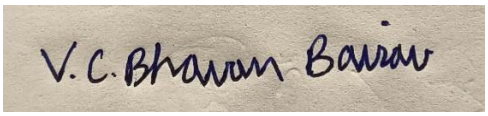
4	A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.	BL4 – Analyze	**5**
5	Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.	BL3 – Apply	**5**
6	An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.	BL4 – Analyze	**5**
7	For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.	BL4 – Analyze	**5**
8	Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.	BL4 – Analyze	**5**
9	A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.	BL4 – Analyze	**5**
10	Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.	BL4 – Analyze	**5**

Code of Conduct:

I V.C.BHAVAN BAIRAV – 192511369 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

QUESTION 1 – E-COMMERCE PRODUCT RECORDS

Problem Analysis:

The company must store 10 million product records. The major requirements are efficient storage, fast retrieval, scalability, and reasonable insertion cost. Product systems commonly perform searches using ProductID, category, brand, price, and other attributes. Since the number of records is very large, sequential searching would require many disk block accesses.

Recommended File Organization:

A **B+ Tree-based indexed file organization** is recommended. Product records can be stored in a clustered or physically organized data file, with B+ Tree indexes on frequently searched attributes.

A heap file provides fast insertion but searching may require scanning many records. A sorted file supports efficient range queries but makes insertion expensive. A B+ Tree provides balanced search performance and supports both equality and range queries.

Strategy	Search	Insertion	Range Query	Suitability
Heap	O(n)	Very Good	Poor	Low
Sorted	O(log n)	Poor	Excellent	Medium
B+ Tree Indexed	O(log n)	Good	Excellent	High

Cost Analysis:

Assume a 4 KB block and an average record size of 200 bytes.

Records per block:

Records per block = FLOOR (4096 / 200)

= 20 records

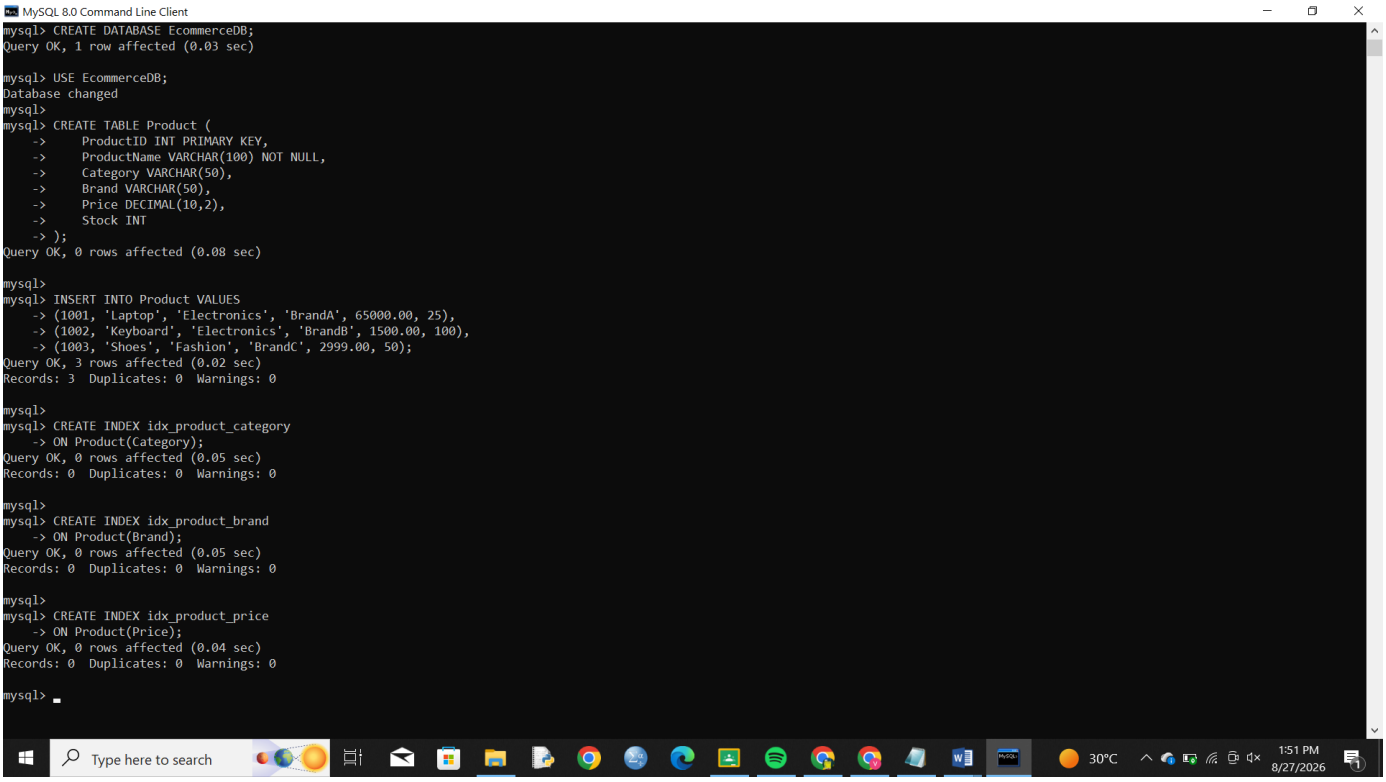
For 10 million records:

Number of data blocks = 10,000,000 / 20

= 500,000 blocks

A heap search may require up to approximately 500,000 block accesses in the worst case. A B+ Tree reduces the search to the tree height plus the required data blocks. The additional index storage is justified because it significantly reduces disk I/O.

SQL Implementation:



Conclusion:

A B+ Tree indexed organization is the most practical choice because it scales to 10 million records, supports equality and range searches, and maintains reasonable update performance.

QUESTION 2 – BANKING TRANSACTION RETRIEVAL

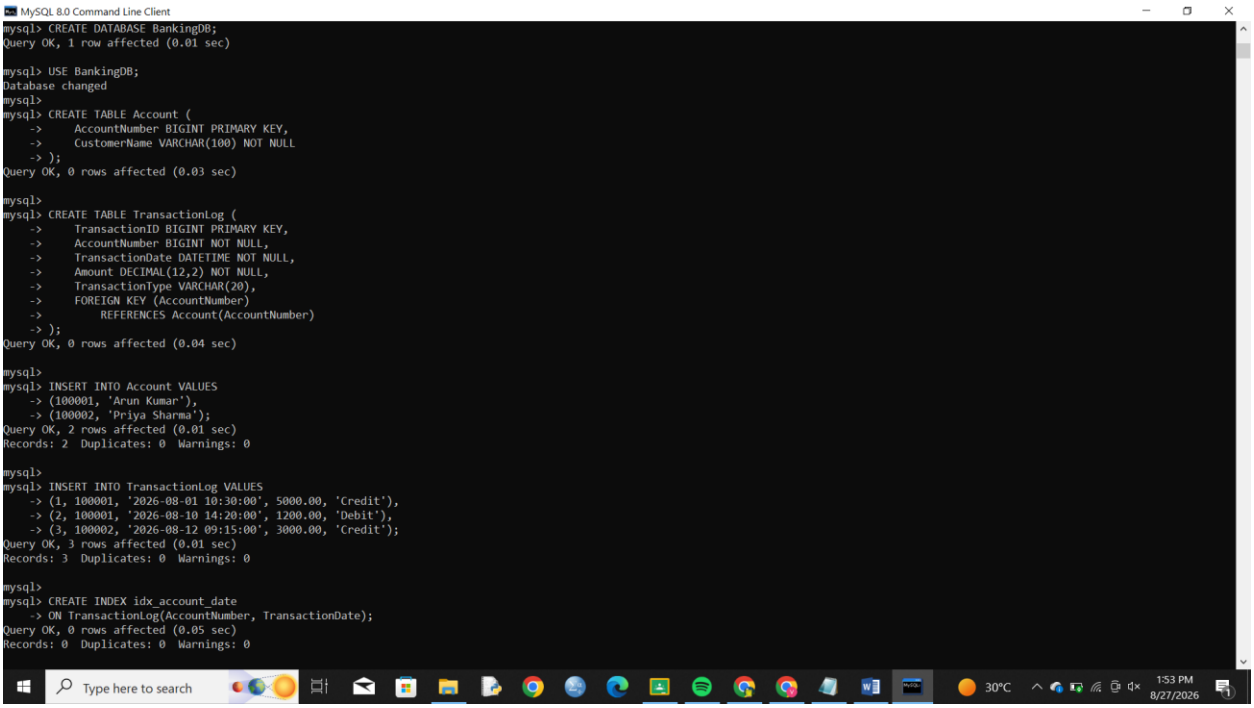
Problem Analysis:

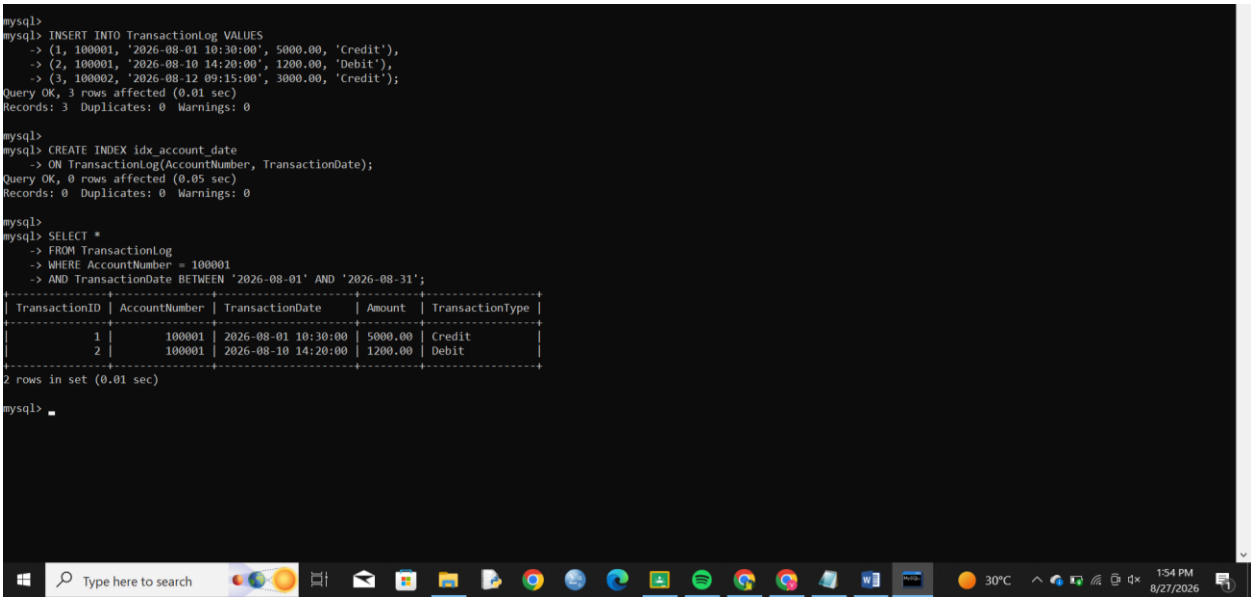
The banking system requires rapid retrieval of transactions for a particular account and within a specific date range. The common query pattern is therefore based on **AccountNumber and TransactionDate**.

Indexing Strategy:

A **composite B+ Tree index** on (AccountNumber, TransactionDate) is recommended. AccountNumber allows direct access to the customer's transactions, while TransactionDate supports an efficient range scan.

SQL Implementation:





Analysis:

The composite index is more efficient than two unrelated indexes for the stated query because both search conditions are represented in one ordered structure. It supports equality searching on AccountNumber followed by a date range scan.

The main trade-off is additional storage and slightly higher write cost whenever transactions are inserted or modified.

Conclusion:

A composite B+ Tree index on AccountNumber and TransactionDate provides fast and scalable transaction retrieval.

QUESTION 3 – HEALTHCARE PATIENT RECORDS

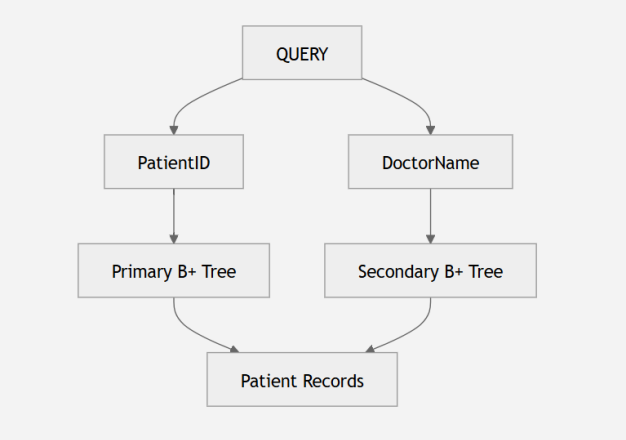
Problem Analysis:

The healthcare system must retrieve patient records using two access paths: **PatientID** and **DoctorName**. PatientID is unique, whereas DoctorName can correspond to many patients.

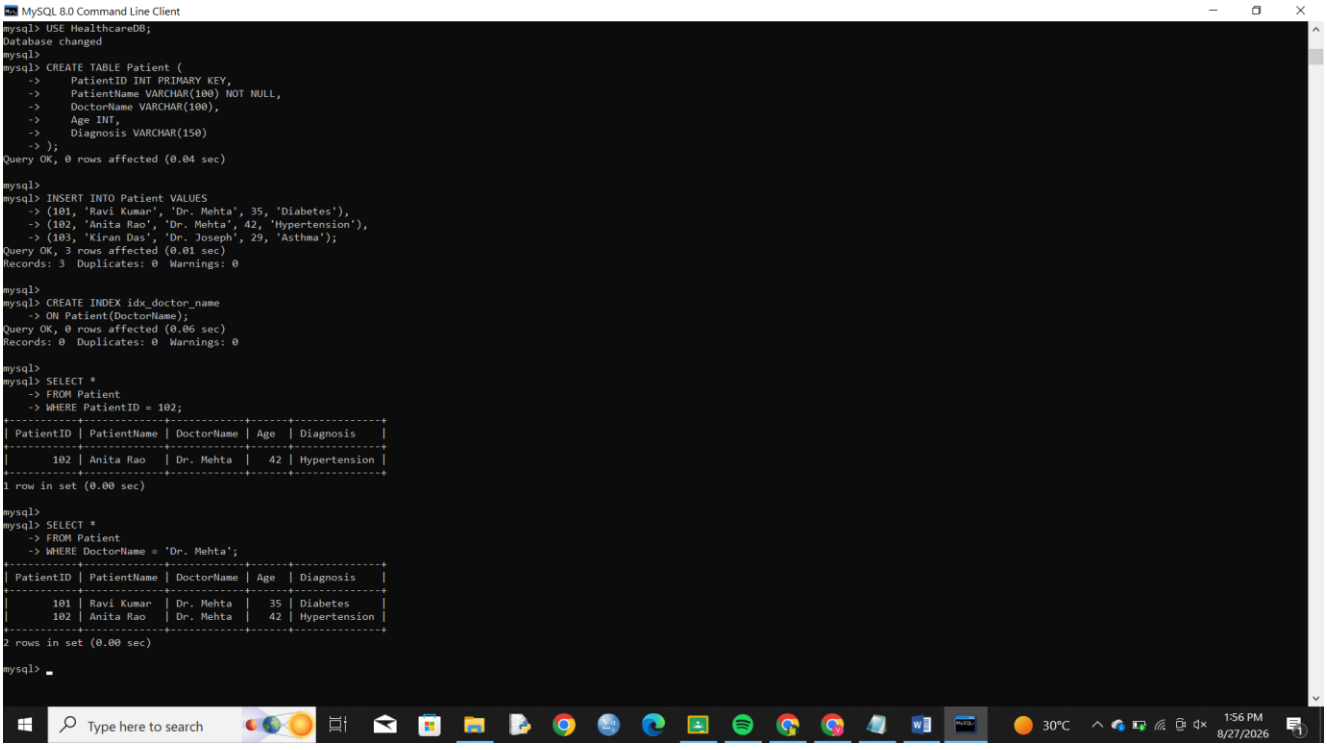
Multi-Level Indexing Design:

A primary B+ Tree index on PatientID provides direct patient retrieval. A secondary B+ Tree index on DoctorName provides doctor-based retrieval.

The access structure can be represented as:



SQL Implementation:



Analysis:

PatientID searches use the primary index, while DoctorName searches use the secondary index. This allows the system to efficiently support both access patterns without performing a complete table scan.

Conclusion:

Separate primary and secondary B+ Tree indexes provide flexible and efficient retrieval for patient records.

QUESTION 4 – SHIPMENT RECORDS: B-TREE V/S B+ TREE

Problem Analysis:

The logistics company experiences frequent insertions and deletions of shipment records. Therefore, the indexing structure must remain balanced and support efficient searching and updating.

Comparison:

Feature	B-Tree	B+ Tree
Data Storage	Internal and leaf nodes	Leaf nodes
Range Queries	Good	Excellent
Sequential Traversal	Less Efficient	Very Efficient
Fan-out	Lower	Higher
Dynamic Updates	Good	Very Good
Database Suitability	Good	Excellent

Recommended Structure:

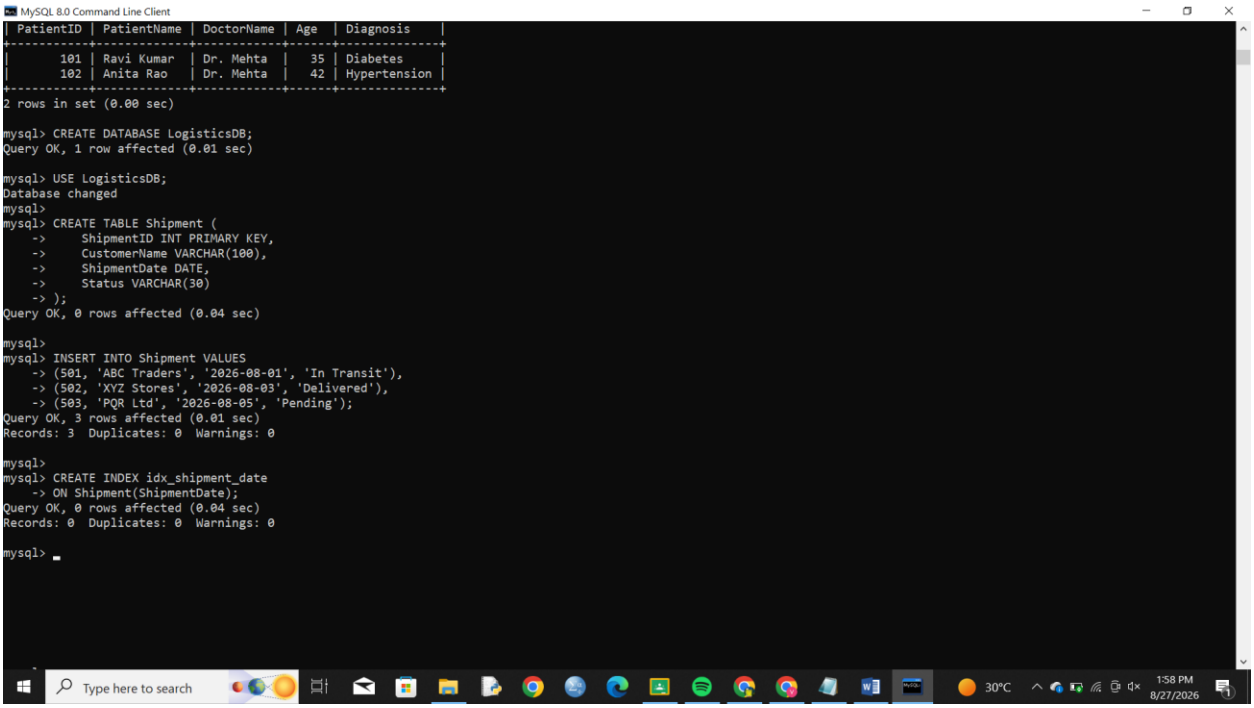
A **B+ Tree** is more suitable for the logistics system.

B+ Tree internal nodes contain keys and pointers, allowing more keys to fit into each index block. This increases fan-out and reduces tree height.

All records are stored at the leaf level, and leaf nodes are linked. This makes range searching and sequential traversal efficient.

Insertions and deletions remain balanced through node splitting and merging.

SQL Implementation:



Conclusion:

For dynamic shipment data, B+ Tree is preferred because of its high fan-out, efficient range access, balanced updates, and excellent sequential traversal.

QUESTION 5 – HR HASHING SCHEME

Problem Analysis:

The HR system contains 5,000 employees. EmployeeID is a suitable hash key because it is unique and frequently used for exact-match retrieval.

Hashing Design:

For static hashing, a bucket count can be selected based on the expected load. For example, using 1,000 buckets gives an average of approximately five records per bucket.

A simple hash function is:

h (EmployeeID) = EmployeeID % 1000

Collisions can be handled using overflow chaining.

Static Hashing Versus Extendible Hashing:

Factor	Static Hashing	Extendible Hashing
Bucket Count	Fixed	Dynamic
Growth Handling	Overflow Pages	Bucket Splitting
Implementation	Simple	More Complex
Scalability	Moderate	High
Suitable For	Stable Dataset	Growing Database

SQL Implementation:

Analysis:

Static hashing is simple and effective when the number of employees is relatively stable. However, extendible hashing is more suitable when the employee database is expected to grow because it dynamically expands buckets and reduces overflow problems.

Conclusion:

For a current 5,000-employee system, static hashing can be used effectively. For future growth, extendible hashing provides better scalability.

QUESTION 6 – AIRLINE RESERVATION SYSTEM

Problem Analysis:

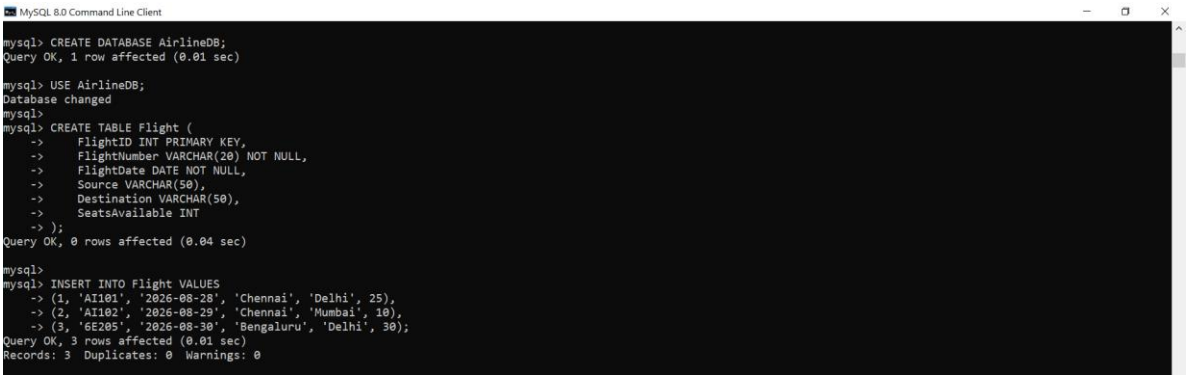
The airline reservation system requires point queries using FlightNumber and range queries using FlightDate. These represent two different access patterns.

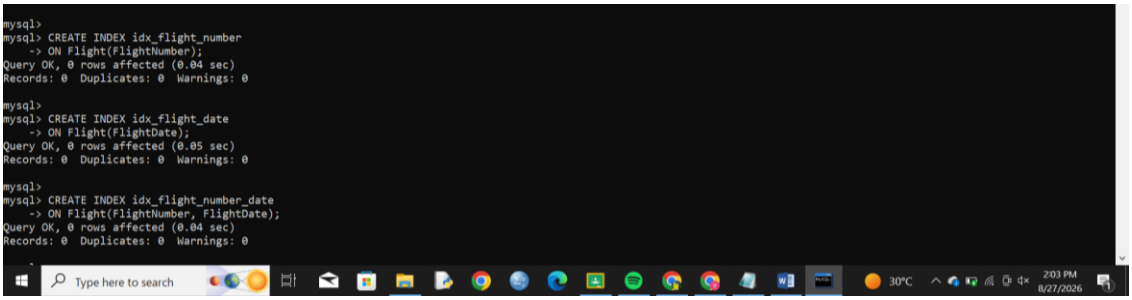
Combined Index Structure:

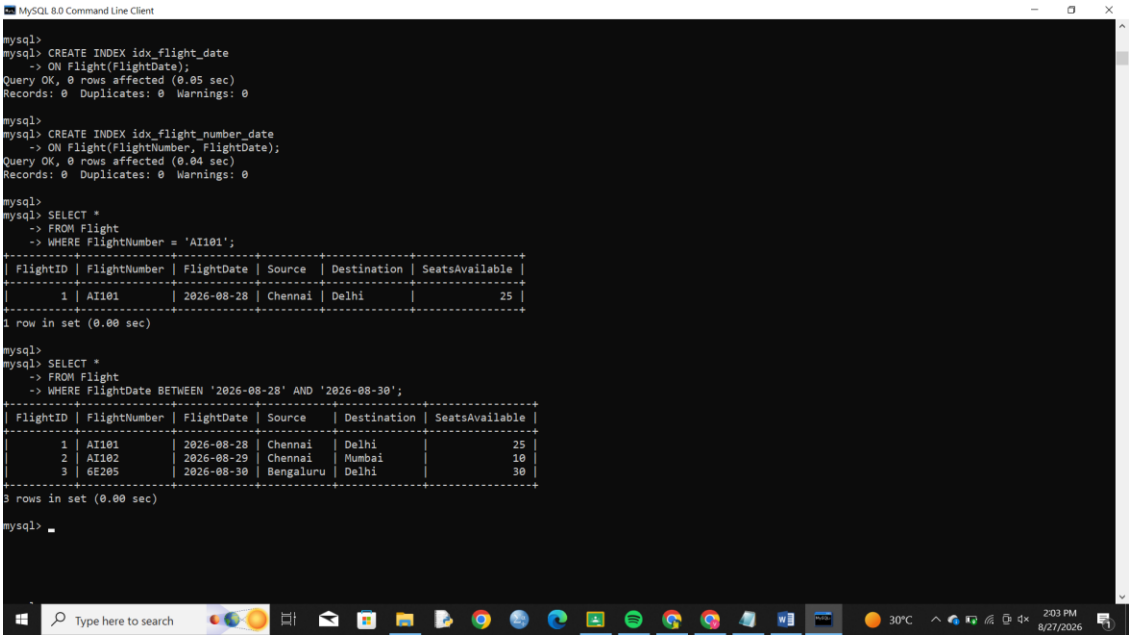
A B+ Tree index on FlightNumber is suitable for point queries. Another B+ Tree index on FlightDate supports range queries.

A composite index on (FlightNumber, FlightDate) can additionally support queries involving a particular flight over a range of dates.

SQL Implementation:







Analysis:

The FlightNumber index provides fast point lookup, while the FlightDate index provides efficient ordered range scanning.

Conclusion:

The combined B+ Tree indexing structure efficiently supports both point and range queries in the airline reservation system.

QUESTION 7 – UNIVERSITY STUDENT DATABASE

Problem Analysis:

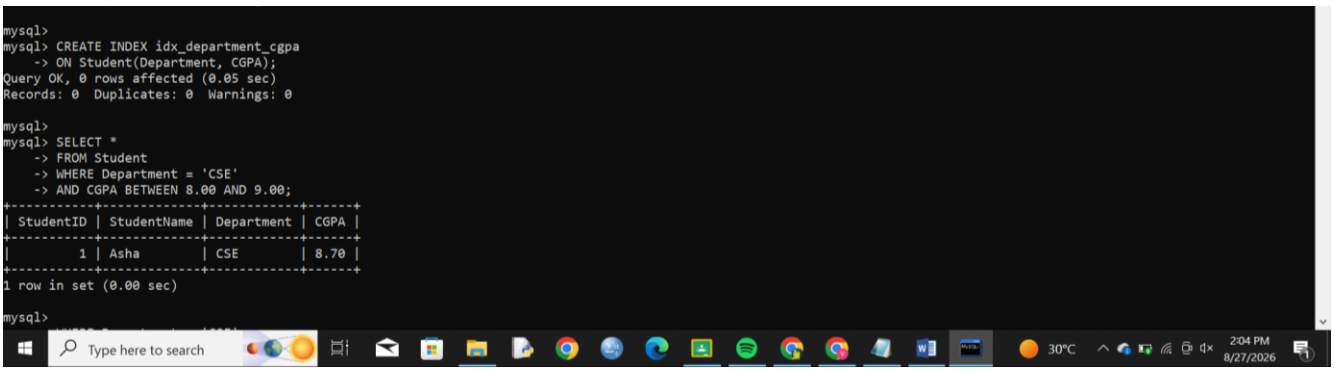
The university database must support queries based on Department and CGPA range. Department is generally an equality condition, while CGPA is a range condition.

File Organization:

Student records can be stored using a heap or clustered organization. Secondary indexes can then provide fast access paths.

If department-based retrieval is dominant, clustering records by Department can further reduce disk I/O.

SQL Implementation:



Analysis:

The composite index first narrows the search to the selected department and then performs an ordered range scan on CGPA.

Conclusion:

A heap or clustered data file with a secondary B+ Tree composite index on (Department, CGPA) provides an efficient and practical solution.

QUESTION 8 – DISK BLOCK UTILIZATION

Problem Analysis:

The system contains 100,000 records. Each record is 50 bytes and each disk block is 4 KB.

Calculation:

Block size = 4 KB
= 4096 bytes
Record size = 50 bytes

Records per block:

= FLOOR(4096 / 50)
= FLOOR(81.92)
= 81 records

Total data size:

= 100,000 × 50
= 5,000,000 bytes

Number of blocks based on complete records:

= CEILING(100,000 / 81)
= 1,235 blocks

Unused space in a full block:

= 4096 - (81 × 50)
= 46 bytes

Comparison:

Organization	Approx. Data Blocks	Utilization	Main Characteristic
Heap	1,235	High	Fast insertion
Sorted	1,235	High	Good range searching
Hashed	1,235 + possible overflow	Depends on load factor	Fast equality search

Analysis:

Heap and sorted organizations require approximately the same number of data blocks because the record size is identical. Their major difference is the organization of records rather than the basic storage requirement.

Hashed organization may require additional overflow blocks when buckets become full.

Conclusion:

For the given fixed-length records, 81 complete records fit into each 4 KB block, requiring approximately **1,235 data blocks**.

QUESTION 9 – SOCIAL MEDIA COMPOSITE INDEXING

Problem Analysis:

The platform stores 500 million posts and needs efficient access based on UserID, Timestamp, and Hashtag.

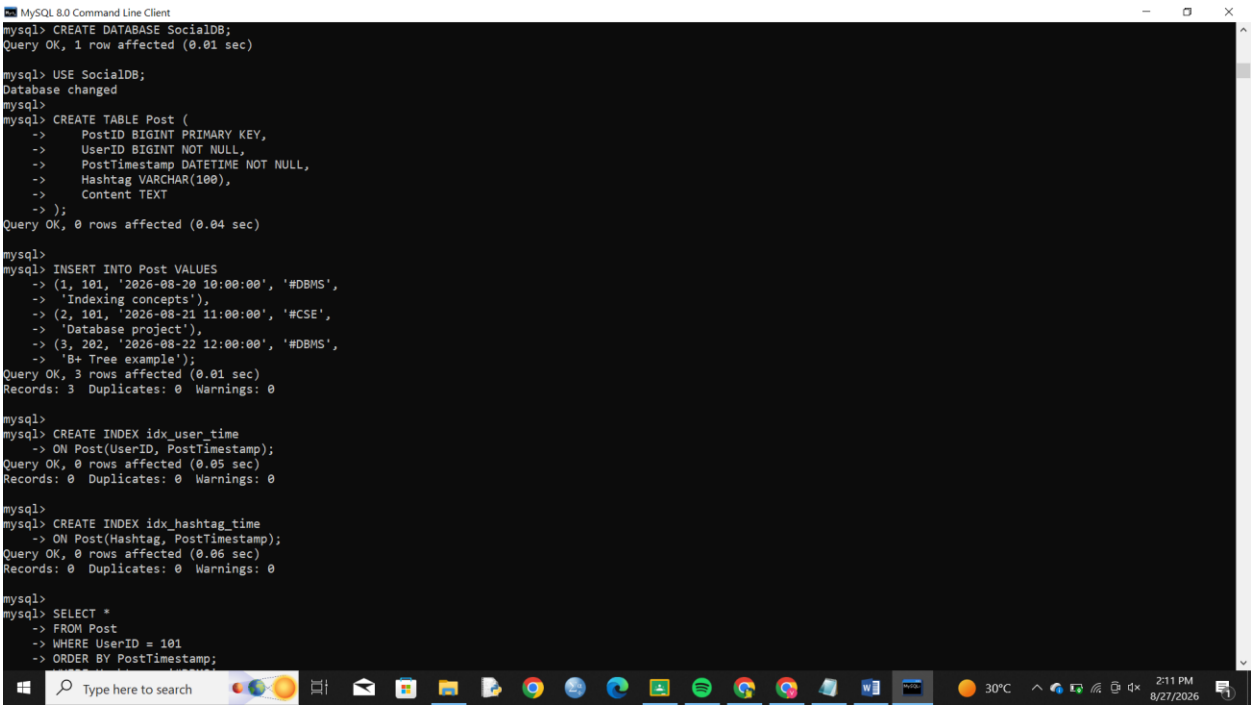
At this scale, excessive indexing increases storage requirements and write overhead. Therefore, indexes should be designed according to actual query patterns.

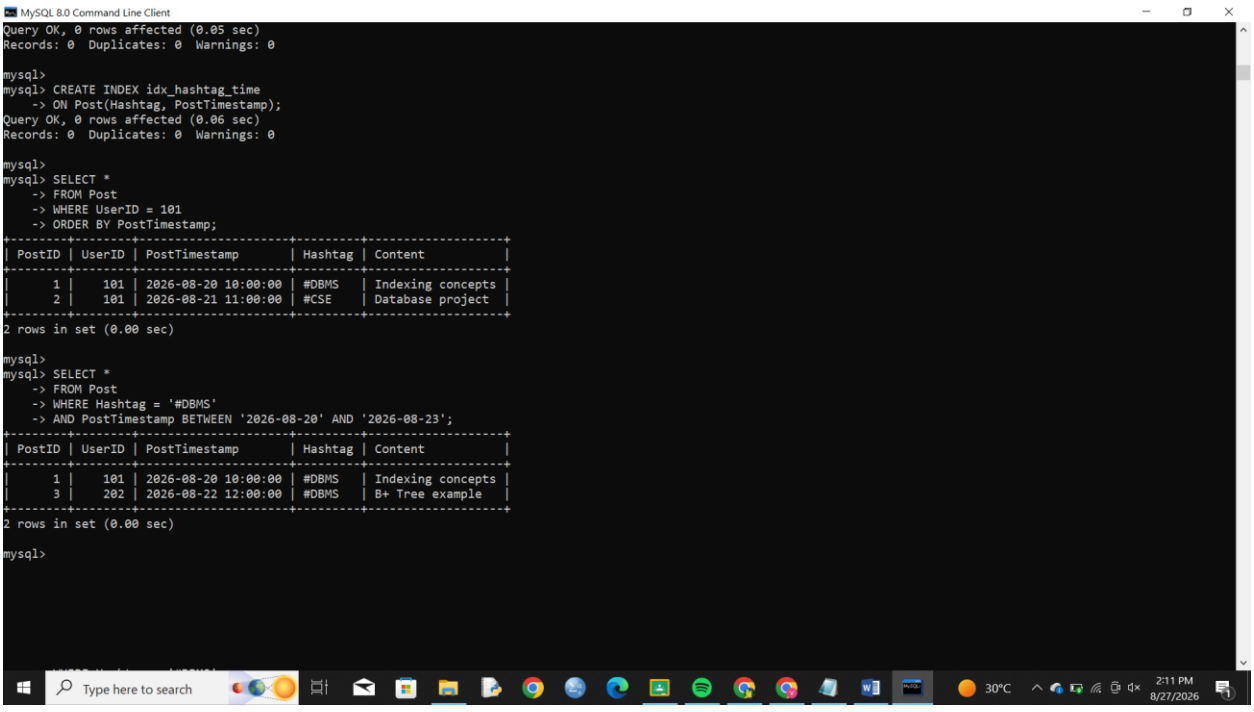
Composite Index Strategy:

A composite B+ Tree index on (UserID, PostTimestamp) is suitable for retrieving a user's posts in chronological order.

Another composite B+ Tree index on (Hashtag, PostTimestamp) supports hashtag searches with date ranges.

SQL Implementation:





Scalability Consideration:

With 500 million records, creating independent indexes on every attribute would create substantial storage and maintenance overhead.

Composite indexes should therefore be selected according to common query patterns.

For very large production systems, time-based partitioning or sharding by UserID can be combined with indexing for further scalability.

Conclusion:

The two targeted composite B+ Tree indexes provide efficient access to user and hashtag queries while avoiding unnecessary indexes.

QUESTION 10 – SUPERMARKET BILLING SYSTEM

Problem Analysis:

The supermarket billing system processes approximately one million transactions daily. It requires efficient insertion of new bills, deletion or archival of old transactions, and fast searching.

Performance Comparison:

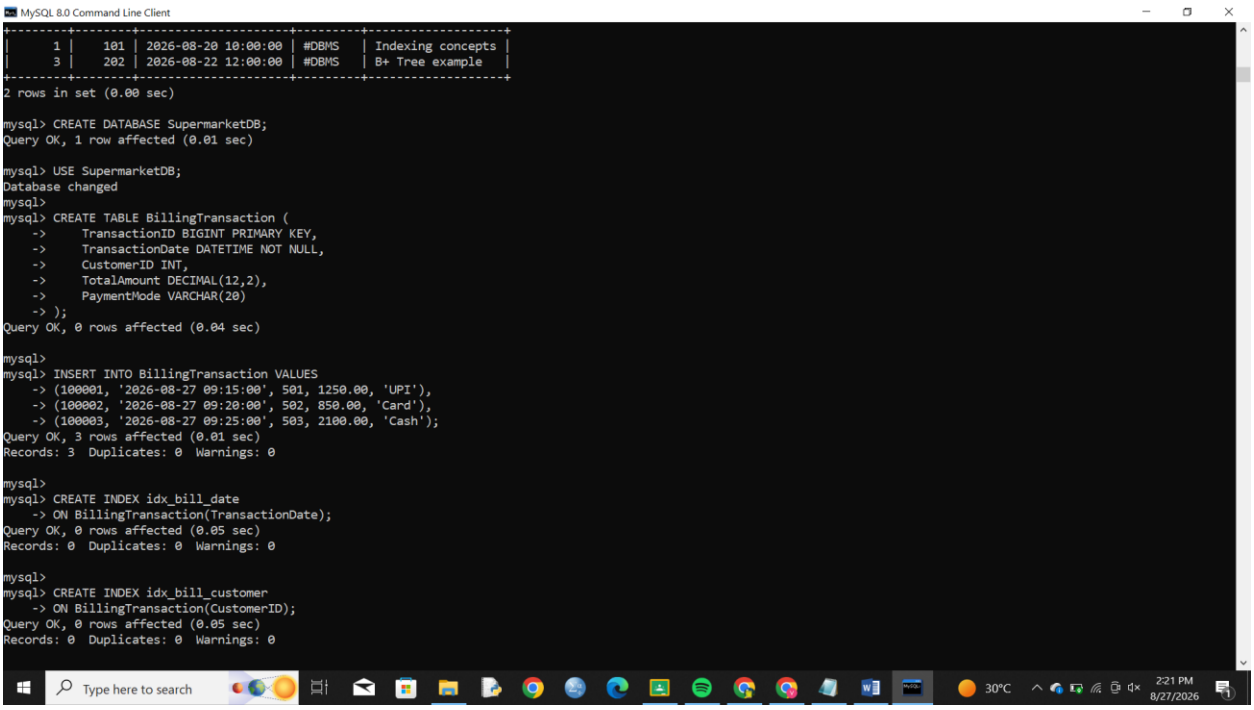
Operation	Heap File	Sorted File	Hashed File
Insert	O(1) Average	O(n)	O(1) Average
Delete	O(n) Without Index	O(n)	O(1) Average
Equality Search	O(n)	O(log n)	O(1) Average
Range Search	O(n)	O(log n + k)	Poor
Best Use	Heavy Writes	Ordered/Range Access	Exact-Match Access

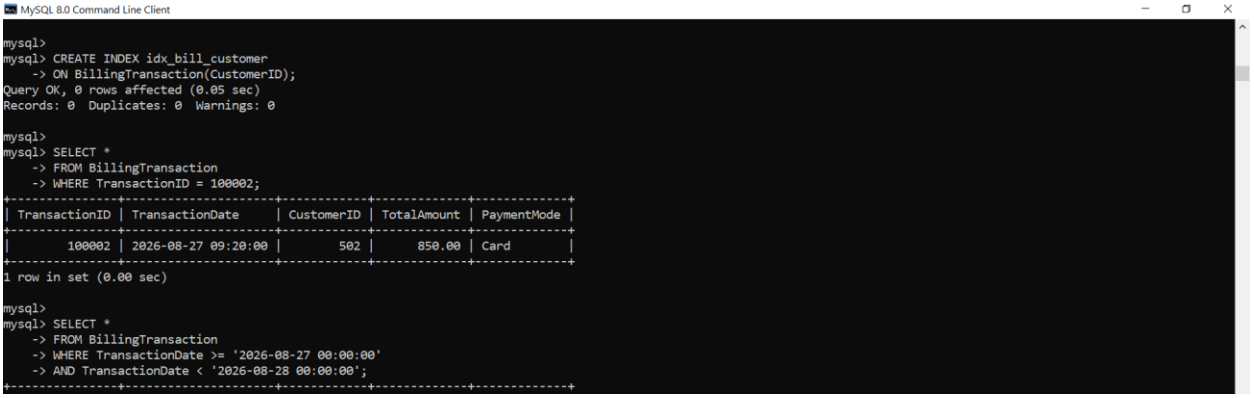
Recommended Design:

A practical supermarket system should use a heap-like append-oriented data organization for high-volume transaction insertion.

B+ Tree indexes can be created on TransactionID and TransactionDate. Hashing can provide fast exact-match lookup, but it should not be the only structure because billing reports frequently require date-range queries.

SQL Implementation:





```
mysql>
mysql> SELECT *
    -> FROM BillingTransaction
    -> WHERE TransactionDate >= '2026-08-27 00:00:00'
    -> AND TransactionDate < '2026-08-28 00:00:00';
+---------------+---------------------+------------+-------------+-------------+
| TransactionID | TransactionDate     | CustomerID | TotalAmount | PaymentMode |
+---------------+---------------------+------------+-------------+-------------+
|        100001 | 2026-08-27 09:15:00 |        501 |     1250.00 | UPI         |
|        100002 | 2026-08-27 09:20:00 |        502 |      850.00 | Card        |
|        100003 | 2026-08-27 09:25:00 |        503 |     2100.00 | Cash        |
+---------------+---------------------+------------+-------------+-------------+
3 rows in set (0.00 sec)

mysql>
mysql>
```

Analysis:

Heap organization provides excellent insertion performance because new transactions can be appended to available data pages.

Sorted organization provides strong searching and range performance but can be expensive for continuous high-volume insertion.

Hashing provides excellent average equality lookup but is weak for range queries.

Therefore, a hybrid design provides the best balance for the supermarket system.

Conclusion:

For one million daily transactions, an append-friendly heap organization with B+ Tree secondary indexes provides a good balance between write throughput, point retrieval, and reporting queries.